

Module 3 — Microbial Genome Assembly

Status: built.

Curriculum module: [Module 3 — Microbial Genome Assembly](#).

Guiding question

What genome sequence can be reconstructed from the reads, and how strong is the evidence for its quality and completeness?

Learning outcomes

- Choose among short-read, long-read, and hybrid assembly strategies based on the actual reads in hand, not a guess.
- Run SPAdes for short reads or Flye for long reads with documented, reproducible parameters.
- Interpret an assembly graph and identify unresolved structures (branch points/repeats, circularity) — not just read the final FASTA.
- Evaluate contiguity (N50/L50 and related statistics) **without** treating N50 as a complete quality measure — the curriculum explicitly flags this as a caveat metric, not a quality guarantee.
- Assess completeness, contamination, and coverage using proxy checks, and know what the real tools (BUSCO/CheckM) would add.
- Distinguish assembly evaluation (this module) from genome annotation (a separate downstream task this module does not cover).

Quality framework (per curriculum)

Dimension	Question students must answer	Addressed by
Contiguity	How fragmented is the assembly, and are large contigs supported?	Script 5
Completeness	Are expected conserved features represented?	Script 6 (proxy) / BUSCO (real tool, documented not run)
Correctness	Are there signs of misassembly, inconsistent coverage, or conflicting read support?	Scripts 4 + 6
Contamination	Do taxonomic or compositional signals suggest mixed material?	Script 6
Fitness for purpose	Is the assembly adequate for typing, gene detection, comparative genomics, or closure?	Script 7 (human judgment call, not auto-generated)

Portfolio artifact (per curriculum)

Assembly FASTA, environment and commands, assembly statistics, graph/coverage evidence, quality interpretation, and a clear fitness-for-purpose conclusion. Script 7 below auto-assembles this as ASSEMBLY_REPORT.md — the topology, statistics, and completeness/contamination sections are filled in automatically from scripts 4-6's own output; the interpretation and fitness-for-purpose sections are

intentionally left blank for the analyst to complete by hand, mirroring Module 2's `DE_REPORT.md` pattern: no script should generate a "this assembly is good" verdict on its own.

Worked-example datasets (real, verified via NCBI eutils)

Unlike this module's stub, which suggested reusing the existing `Final_master_end_to_end_amr.sh` AMR pipeline's internal SPAdes step, this module uses its own dedicated worked example so assembly can be taught as a standalone skill, separate from AMR gene calling:

Read type	Accession	Organism	Platform	BioProject	BioSample	SRA study	Size	Reads/spots
Short-read	SRR1770413	<i>E. coli</i> K-12	Illumina MiSeq, paired-end, ~600 bp insert	PRJNA272917	SAMN03287706	SRP052773	~189 MB	643,253 spots
Long-read	SRR39619343	<i>E. coli</i> K-12 MG1655	Oxford Nanopore PromethION	PRJNA1490942	SAMN61464373	SRP717896	~795 MB	125,100 spots (avg length 9,782 bp)

Both accessions are the *same* organism (*E. coli* K-12) sequenced with two different technologies, so the same genome can be assembled with script 2 (short-read) and script 3 (long-read) and the two results directly compared — the pedagogical point of the strategy-choice framework in script 1.

Data-integrity note (open finding, out of scope to fix here)

This repository's existing `srr_list_for_final_master_end_to_end_amr.sh` hardcodes `SRR11092056/SRR11092057` with comments assuming a *Salmonella*/bacterial isolate WGS run. Verifying via NCBI `efetch`, these accessions are actually **SARS-CoV-2 metagenomic RNA-Seq** data (BioProject `PRJNA605983`, Wuhan Institute of Virology, strain `WIV06`, TaxID `2697049`) — not a bacterial isolate at all. This is flagged here as an open data-provenance note; per this project's "conserve what's already delivered" policy, that script is **not** modified in this session (it belongs to the AMR/supplemental-case-studies module, out of this module's scope) — but anyone using it for a genome-assembly exercise should substitute a real bacterial accession such as the two above.

Scripts in this module

Seven scripts, each adding exactly one new idea on top of the previous one:

#	File	New idea introduced
1	<code>scripts/1_Choose_Assembly_Strategy.sh</code>	Sample actual read lengths from the FASTQ file(s) provided and programmatically recommend <code>SHORT_READ</code> / <code>LONG_READ</code> / <code>HYBRID</code> / <code>UNCERTAIN</code> — replacing "assume you already know which assembler to run" with an evidence-based decision step.
2	<code>scripts/2_Run_SPAdes_Short_Read_Assembly.sh</code>	Run SPAdes with a documented, named-flag interface (<code>--threads</code> , <code>--mem</code> , <code>--hybrid</code>) and confirm the graph file (<code>assembly_graph_with_scaffolds.gfa</code>) is preserved alongside the FASTA — not just the contigs, which is all most tutorials keep.

#	File	New idea introduced
3	<u>scripts/3_Run_Flye_Long_Read_Assembly.sh</u>	Run Flye with a documented <code>--low-mem</code> flag, and teach the real, source-verified reason a memory-constrained machine can crash: Flye's default <code>--nano-raw</code> k-mer size ($k=17$) allocates a fixed ~8.6 GB flat array ($4^{17}/2$ bytes) for k-mer counting regardless of genome size — confirmed by rebuilding Flye from source and tracing the crash to <code>KmerCounter::count</code> in <code>vertex_index.cpp</code> . <code>--low-mem</code> uses <code>kmer_size=15</code> (~537 MB) as a documented, verified workaround; real HPC nodes with sufficient RAM should keep Flye's tuned default $k=17$ for better repeat resolution.
4	<u>scripts/4_Interpret_Assembly_Graph.py</u>	Parse the GFA graph directly (pure Python, no dependency) to detect branch points (node-ends touched by more than one <i>distinct</i> edge — the signature of an unresolved repeat) and circularity (a true self-loop edge) as two separate, non-conflicting signals — a circular single-contig genome is correctly reported as 0 branch points + 1 circular contig, not miscounted as a false branch.
5	<u>scripts/5_Compute_Assembly_Statistics.py</u>	Compute total length, contig count, N50/L50, N90/L90, longest/shortest contig, and per-contig GC% — and print the curriculum-mandated caveat every single run : N50 is a contiguity statistic, not a correctness/completeness/contamination guarantee.
6	<u>scripts/6_Assess_Completeness_And_Contamination.py</u>	Two independent, dependency-light proxy checks: per-contig GC% vs. the assembly's length-weighted mean (not a naive per-contig average, which a short outlier contig can skew), and per-contig read depth (via <code>minimap2+samtools</code>) vs. the assembly's length-weighted median. Contigs flagged by both checks are the strongest contamination signal. Documents the real tools (BUSCO, CheckM) and their exact commands as the fallback not run here — mirrors Module 2's <code>apeglm</code> -fallback documentation pattern.
7	<u>scripts/7_Assembly_Quality_Report_And_Fitness_For_Purpose.py</u>	Auto-assembles <code>ASSEMBLY_REPORT.md</code> from scripts 4-6's own output, with the objective sections filled in automatically and the interpretation / fitness-for-purpose sections left as explicit blank prompts — and a closing section that explicitly separates assembly evaluation from genome annotation (a different downstream task this module does not cover).

All seven scripts were syntax-checked (`bash -n` for the shell scripts, `python3 -m py_compile/ast.parse` for the Python scripts) and **functionally tested end-to-end** against synthetic fixtures built specifically to exercise every check:

- A 49,400 bp synthetic genome with a deliberately duplicated 1,200 bp repeat block (forces a real branch point in the SPAdes graph).
- A 4,000 bp synthetic contaminant sequence built at ~82% GC vs. the genome's ~50% GC, mixed into the short-read fixture at low relative coverage (~3x contaminant vs. ~35x genome) — correctly flagged by **both** the GC and depth checks in script 6 once assembled.
- A clean 200,000 bp synthetic genome (no engineered repeat) for the long-read/Flye path, which correctly assembles to one circular contig with 0 branch points.
- Script 4 was verified against both real graph shapes: the SPAdes graph (branch point present, linear/non-circular) and the Flye graph (0 branch points, 1 circular contig).

- Script 6's depth check was corrected mid-testing to use a **length-weighted** median instead of a naive per-contig median — with only 5 contigs (1 large genome contig + 4 small contaminant fragments), an unweighted median is dominated by contig *count*, not by how much of the assembly each contig represents, and had originally flagged the correct majority contig as the "outlier" while missing the real contaminant fragments. The fix and the reasoning are documented inline in the script.

How to run

1. Run script 1 on your FASTQ file(s) to get a strategy recommendation:

```
bash 1_Choose_Assembly_Strategy.sh --reads1 reads_1.fastq[.gz] [--reads2
reads_2.fastq[.gz]]
```

2. Run the recommended assembler:

- Short-read: `bash 2_Run_SPAdes_Short_Read_Assembly.sh reads_1.fastq reads_2.fastq spades_out --threads 8 --mem 32`
- Long-read: `bash 3_Run_Flye_Long_Read_Assembly.sh reads.fastq flye_out --threads 8 [-low-mem]`

3. Interpret the graph: `python3 4_Interpret_Assembly_Graph.py`

```
<assembler_out>/assembly_graph*.gfa
```

4. Compute statistics: `python3 5_Compute_Assembly_Statistics.py <assembler_out>/contigs.fasta --label "SPAdes" (or assembly.fasta --label "Flye")`

5. Check completeness/contamination: `python3 6_Assess_Completeness_And_Contamination.py <assembler_out>/contigs.fasta --reads1 reads_1.fastq --reads2 reads_2.fastq`

6. Assemble the report: `python3 7_Assembly_Quality_Report_And_Fitness_For_Purpose.py <assembler_out>/contigs.fasta <assembler_out>/assembly_graph*.gfa --reads1 reads_1.fastq --reads2 reads_2.fastq --assembler "SPAdes" --out ASSEMBLY_REPORT.md`

7. Open ASSEMBLY_REPORT.md and fill in sections 4 (interpretation) and 5 (fitness-for-purpose conclusion) by hand — the intentional human-judgment step the curriculum requires.

See [RUNBOOK.md](#) for the full step-by-step execution order, required tools, expected outputs, and troubleshooting.

What this replaces / extends

The stub previously pointed at [modules/supplemental-case-studies/scripts/Final_master_end_to_end_amr.sh](#), which runs a SPAdes step internally as part of AMR gene calling but does not expose assembly as a standalone skill. This module's scripts 1-7 extract and extend that idea into a dedicated, assembler-agnostic (short-read/long-read/hybrid), graph-aware, quality-framework-driven exercise, while leaving the AMR pipeline itself untouched.

What this module does not do (scope note)

This module evaluates whether a genome was assembled well — it does not annotate the resulting sequence (identify genes, operons, or function within it). Annotation is a separate downstream task (typical tools: Prokka, Bakta, NCBI PGAP) that assumes the assembly underneath it is already trusted. See script 7's closing report section for this same distinction spelled out for students.

Teaching materials

- Lecture deck: [lecture/Genome_Assembly_Lecture.pptx](#).
- Runbook: [RUNBOOK.md](#).
- Still to develop per [Section 5 of the curriculum](#): guided student workbook, dataset card/provenance record for SRR1770413/SRR39619343, and a transfer-task assessment.